
> この　は、これからこの「　　（こうすう）システム」を引き　ぐあなたが、　　です。まだ1　もコードを　く　　はありません。まずは「このアプリは何のために　られたのか」「ブラウザに　　された画　の　で、いったい何が　きているのか」「ファイルがどこに何のために　いてあるのか」を、　　つかんでいきます。
>
> 　にこの　の　　は、　　の　　を、フロントエンド（`Login.tsx`）→ ルーティング（`urls.py`）→ サーバー　　（`main_views.py`）→ データベース　　（`models.py`）と、　　ことです。1　の　　を　　から　　まで　して　れば、「Webアプリってこう　くのか」が　の　から　かります。
>
> この　は　いですが、　　「こういう　　があったな」と　い　せるように、　　のように何　も　ってきてください。

- 　　— 　　とは何か。なぜこのアプリが　　なのか
- 　　— ブラウザ・サーバー・データベースの　割　　を、　　やレストランにたとえて　解する
- 　　— なぜ画　（React）と　　（Django）を　々に　るのか
- 　　— `Login.tsx` → `kosu/urls.py` → `main_views.py` → `models.py` を1　ずつ
- 　　— React・TypeScript・Django・PostgreSQL を「7つの　　」で　価する
- 　　— アプリの「　　ファイル」の　　を　み解く
- 　　— どのURLがどこへ　り　けられるのか
- 　　— どの画　が、どのURLで　　されるのか
- 　　— どこに何があるのか

0. はじめに：この　の　み
1. このアプリは何をするものか（　　とは）
2. Webアプリの3　：ブラウザ・サーバー・データベース
3. フロントエンドとバックエンドの
4. 　　：1回のログインで何が　きるか（　　）
5. 　コード　：Login.tsx（フロントの　　）を1　ずつ
6. 　コード　：kosu/urls.py（URLの　　）を1　ずつ
7. 　コード　：main_views.py の Login（サーバーの　　）を1　ずつ
8. 　コード　：models.py（データの　　）を1　ずつ
9. 　コード　：hozen_another/urls.py（　　位のルーター）を1　ずつ
10. 　コード　：index.tsx（フロントの　ルート）を1　ずつ
11. 　　スタックと　　（7　　で　価）
12. 　コード　：settings.py の　　を1　ずつ
13. フォルダ　　の
14. トラブルシューティング
15. 演
16. この　のまとめ

この　　は、　　に　けて　かれています。　　が　てきたら、その　で「　　（よみがな：意　）」の　で　　します。たとえ　も　　します。

> 　　がテキストエディタで　いた、プログラムの「　　そのもの」のこと。`.tsx` `.py` `.css` などの拡　　のファイルに　かれた　　です。これをコンピュータが　み　って　　することで、画　が　きます。　　では、このソースコードを　　引　して解　します。

>

> - … ユーザーが　　て　る「画　」　。　アプリでは　　で　られています。お　でいう「お　さんが　る　り　」。
> - … 画　の　で、データを　　したり　　したりする「　　」　。　アプリでは　　で　られています。お　でいう「　の　・　・　」。

この　では、まず概　を　　し、その　で　　み　ます。コードの引　箇　には　ず「これは　ファイルの〇　　」と　かるように　します。

> 「ある　　に、どれだけの　　を　ったか」を　す　のこと。たとえば「1　が1　　かけた　　」を　　に、「この　　には3　　（にんじ）かかった」のように　えます。　　や　　（ほぜん：　　の　・　　）の　　では、「どの　　にどれだけ　　をかけたか」を　　して、ムダを　つけたり、　員　画を　てたりするのに　います。

このアプリは、ひとことで　えば　　です。プロジェクトの　　（`CLAUDE.md`）にもこう　かれています。

> This is a work hours (kosu) management system for tracking employee work time and task categories.（これは、　　員の　　と　　を　　するための、　　システムです。）

ほど §10 で　コードを　ますが、このアプリには　のような画　（ページ）があります。　に　をつかんでおきましょう。

	員　で　り、各　へ案　する	`MainPage/`
	その　の　・　を　、一　・　・カレンダー	`KosuPage/`
	員（　員）の　・一　・　・	`MemberPage/`
	チーム（　）の　と、　員の　の確	`TeamPage/`
	「どんな　があるか」の　を	`DefinitionPage/`
	・　・　い　わせの	`InquirPage/`
	・　・　の確	`AdministratorPage/`

> 「　の　」は会　や　によって違い、また　によって　わります。それを　ことになり　です。そこで「　の一　」を　できるようにしてあります（`kosu_division` モデル。§8で　）。これは「　を、プログラムから　りす」という、よい　の　です。

- 　…　の　を　する（　なし）
- 　…　に加え、　員の　を確　する（`authority` =　あり）
- 　…　の　、　の　、　確　ができる（`administrator` =　フラグあり）

この「3　階の　」は、　で `member`（　員）モデルの `authority`（　）と `administrator`（　）という2つの　で　されているのを　ます（§8）。サーバー　の（`main_views.py`）でも、`if not member_data.authority:` のように　チェックが　に　っています（§7で　）。

- … ブラウザ → サーバー へ　る「お　い」。「この　　をくれ」「このデータを　してくれ」など。
- … サーバー → ブラウザ へ　す「　　」。お　いの　果（　　・　　・データ　）が　っています。この「お　い→　　」が、Webアプリの　です。　アプリでも、画　を開くたび・ボタンを押すたびに、このやり　りが何　も　しています。

ブラウザとサーバーが会　するときの「　のルール（プロトコル：　　）」。HyperText Transfer Protocol の　。`http://localhost:8000` の `http` がこれです。リクエストには「　　」という　があり、　アプリでは　に　の4つを　います。

リクエストの「　　」を　すラベルです。レストランの　　の　　だと　ってください。

- … 「データをください」（一　を　る、画　を開く）。　：メニュー画　を　。
- … 「　しいデータを　ります」（　　、ログイン）。　：ログイン　　を　る。
- … 「　　のデータを　き　えます」（　　）。　：　　の　　。
- … 「このデータを　してください」（　　）。　：　　の　　。§7 で　る `main_views.py` には、`def post(self, request):`（POSTのとき）、`def get(self, request):`（GETのとき）のように、　　かれています。

開　　、あなたのPCの　でブラウザもサーバーも　かします。そのとき　てくる　　が `localhost`（ローカルホスト：このPC　　）です。

- フロントエンド（React開　サーバー）… `http://localhost:3000`
- バックエンド（Django）… `http://localhost:8000`

:3000 や :8000 の 。1 のPCの で、 のプログラムが に できるように ける「 屋 」のようなもの。 じPC（localhost）でも、3000 の 屋にはReact、8000 の 屋にはDjangoが んでいる、というイメージです。なぜ で けるかは、 で settings.py の CORS （コルス） （§12）で してきます。

ながらのWebアプリは、サーバーが「完 したHTMLページ」を 回まるごと ってブラウザに っていました。 アプリは違い、
います。

```
【昔の方式】
ブラウザ ―「ページちょうだい」―> サーバー ―> HTMLを丸ごと み てて す
  ボタンを押すたびに、画 が真っ白になって、全 み み直し

【本アプリの方式 フロント・バック分 】
ブラウザ(React) ―「データだけちょうだい JSON 」―> サーバー(Django)
                <―「はい、データです JSON 」――――
  Reactが受け取ったデータで、画 の必 な 分だけを書き換える。画 は真っ白にならない
```

JavaScript Object Notation の 。データを「 : 」の ペアの まりで す、 の き 。 にも みやすく、 械も解 しやすいので、フロントとバックの でデータをやり りする「 」として われます。たとえばログイン 、サーバーは のようなJSONを します（§7で を ます）。

```
{"status": "success"}
```

「status（ ）という の が success（ ）だ」という意 です。

	「画　の　た　」は `frontend/` だけ、「　　・データ」は `kosu/` だけ、と　す　がはっきりする
	データだけ　け　り、　　な　　だけ　き　えるので、画　　の　　み　みがない
	「これは　た　の　　（フロント）か、データの　　（バック）か」で　因を　り　けやすい
	じバックエンドAPIを、　の画　やアプリからも　べる

> 　　バグが　たとき、まず「これは画　（フロント）の　　か、　　（バック）の　か」を　り　けると、　　がぐっと　くなります。
> 　　、　　が　　です。

> 　　Application Programming Interface の　。　しそうですが、ここでは　　くらいに　えてください。　アプリでは、`/api/login/` `/api/main_menu/` のように、`/api/` で　まるURLが「バックエンドへの　　」です。フロントは、この　にリクエストを　り、JSONで　を　け　ります。

つまり　アプリの　　は、こうです。

```
React　frontend/　──/api/oo/へリクエスト──> Django　kosu/　──> PostgreSQL
　　た目・操作　　　　JSONでやり取り　　　　処理・判断　　　保存
```

この「`/api/` が　　」という　　は、　で `hozen_another/urls.py`（§9）に　コードで　かれているのを確　します。

ここからが　の核　です。　を　に、　の
ファイルをまたいで　を　いかけます。まず　の　れを　で　て、その　§5〜§8 で各ファイルを1　ずつ　みます。

> 　ユーザーが　員　を　し「ログイン」ボタンを押す（`Login.tsx`）→　Reactが `/api/login/` へ、　を　れたPOSTリクエストを　る（`Login.tsx`）→　Djangoが「`/api/login/` は　が　？」とURL　を引く（`hozen_another/urls.py` → `kosu/urls.py`）→　の `Login` クラスの `post`　が　く（`main_views.py`）→　「その　は　員データにあるか？」をデータベースに　い　わせる（`models.py` の `member`　）→　あれば「　」、なければ「エラー」のJSONを　す（`main_views.py`）→　Reactが　を　て、　ならメインメニューへ画　移、　ならエラー　（`Login.tsx` → `index.tsx`）

```
[ブラウザ] Login.tsx
   │ ① 従業員番号 12345 を入力 → ログインボタン押下
   │ ② api.post("/api/login/", { employee_no: 12345 })
   ▼
[サーバー] Django
   │ ③ hozen_another/urls.py が「/api/ → kosu.urls」、kosu/urls.py が「login/ → Login」と振り分け
   │ ④ main_views.py の Login.post() が実　される
   │ ⑤ member.objects.filter(employee_no=12345).exists() で
   ▼　　データベースに「この番号いる　」と問い合わせ
[DB] PostgreSQL　models.py で　　した member
   │ ⑥ 「いる」or「いない」を　す
   ▼
[サーバー] Django
   │ ⑦ いれば {"status":"success"} / いなければ {"status":"error", ...}
   ▼
[ブラウザ] Login.tsx
      ⑧ success なら navigate("/")　メインメニューへ　/ error ならエラー
      ⑨ index.tsx のルート定　が「/」→ MainMenu を
```

この1　の　　を完　に　解できれば、　アプリの　　で　いている
ことが　かります。　　も、　員　も、　　も、「フロントがリクエスト → urlsが　り　け →

viewsが　　→ modelsの　を　み　き → JSONで　す」という　じ　格です。

それでは　　のコードを　みます。　　は、ユーザーが　　に　れる画
`frontend/src/MainPage/Login.tsx`（　71　）です。

> この　では　　に　　します。画　の　た　（JSXとCSS）の　しい解　は `03_Web基　_HTML_CSS.md` にあります。ここでは「ログインの　　が、どこからどこへつながっているか」を　　に　みます。

> 　　このファイルが　う「　　（　　やライブラリ）」を、　のファイルから　り　せます。　　でいう「　と　　を、　から　　に　す」　です。

```
// ① React本体と、状態　理の　具useStateを取り寄せ
import React, { useState } from "react";
// ② 画　リンクと、画　　動の　具を取り寄せ
import { Link, useNavigate } from "react-router-dom";
// ③ 自作の「API へ　信する　前の　具(axios　定済み)
import api from "../api/axios";
// ④　信ライブラリ本体(エラー判定に使う)
```

- `api` は、このアプリ　に　　みの　　です。`../api/axios` というファイルで「サーバーの　」や「クッキーを　る　」がまとめてあり、　回　かずに　みます。
- `axios`（アクシオス）　。ここでは　に「これは　エラーかどうか」を　する `axios.isAxiosError` に　います。

> 「　ファイルで　意された　を、このファイルに　りむ」　。`from "..."` の　が「どのファイルから」を　します。`"react"` のように　だけのものは　（外　から　れた　）、`"../api/axios"` のように `./` `../` で　まるものは　です（`../` は「1つ　のフォルダ」の意　）。

> 「よく　う　を、　かがまとめて　って　開してくれた　」。　でゼロから　らず、　り　せて　います。`React`・`axios`・`react-router-dom` はすべてライブラリです。　（library）から　を　りるように、　を　りて　うイメージ。

> ログイン画　という「　（コンポーネント）」を　し、その　で「　された　員　」と「エラーメッセージ」を覚えておくための（　）を　意します。

```
// ① Loginという画　品を定　（　数コンポーネント）
const Login: React.FC = () => {
  // ② 入力された従業員番号を　える　（初期値は　文字）
  const [employee_no, setNumber] = useState<string>("");
  // ③ エラーメッセージを　える　（初期値は　文字）
  const [errorMessage, setErrorMessage] = useState<string>("");
  // ④ 画　動の　具を使えるように準備
  const navigate = useNavigate();
```

- `const Login: React.FC = () => { ... }` は「`Login` という　の　（画　）を　る」という　。`React.FC` は「これはReactの　コンポーネントです」というの　（TypeScriptのしるし）。
- `useState`（　フック）が、この　でいちばん　な　です。

画　を　　する「　　」のこと。ボタン1つ、フォーム1つ、画　1　を、それぞれ　　した　　として　り、　み　わせて　　を　ります。レゴブロックのイメージ。`Login` は「ログイン画　まるごと1　　」です。

とは「画　が覚えておく、　化する　」のこと。ここでは「いま　　に何が　っているか（`employee_no`）」「エラーは　ているか（`errorMessage`）」を覚えています。`const [値, 値を変える　数] = useState<型>(初期値);` という　で　います。

- `employee_no` …　の　（　むときに　う）
- `setNumber` …　を　えるための　　（　き　えるときに　う）
- `useState<string>("")` …「　　(string)　の　　を、　　は　　`""` で　る」

Reactでは、stateを `setNumber(...)` で　き　えると、　　からです。　　の　　を　き　えても、Reactは　づかず画　が　　されません。「stateを　える＝画　を　　する　　」と覚えてください。

ここがこの画　の　　です。ボタンを押したときに　　される　　を、　　に　みます。

「ログイン」ボタンが押されたら、(1) 画　の　　を　め、(2)　いエラー　　を　し、(3)　　された　　をサーバーの `/api/login/` へ　り、(4)　　が「　　」ならメインメニューへ移　、(5)　　ならエラーメッセージを　　する——という一　の　れを　います。

```
  // ①　信時に呼ばれる　数(asyncは「待つ処理」を含む印)
  const handleSubmit = async (event: React.FormEvent) => {
    // ② ブラウザ標準の「ページ再　み　み」を止める
    event.preventDefault();
    // ③ 前回のエラー　　を消す(　文字にする)
    setErrorMessage("");

    // ④ ここから「失敗するかもしれない処理」
    try {
      // ⑤ サーバーへ番号を　る
      const response = await api.post("/api/login/", { employee_no: Number(employee_no)
});
      // ⑥ サーバーからの　事の中　(JSON)を取り出す
      const data = response.data;
      // ⑦　事が「成功」だったら
      if (data.status === "success") {
        // ⑧ メインメニュー("/")へ画　　動
        navigate("/");
      // ⑨ 「成功」以外だったら
      } else {
        // ⑩ サーバーのメッセージを
        setErrorMessage(data.message || "サーバーエラーが発生しました。");
      }
    // ⑪　信そのものが失敗したとき(例外発生)
    } catch (err) {
      // ⑫ それが　信由来のエラーなら
      if (axios.isAxiosError(err)) {
        // ⑬ 401(未　　)なら、ログイン画　へ戻す
        if (err.response?.status === 401) navigate("/login");
        // ⑭ それ以外は、サーバーのメッセージを
        else setErrorMessage(err.response?.data.message);
      // ⑮　信以外の　のエラー
      } else setErrorMessage("不明なエラーが発生しました。IT担当　に　　してくださ
い。");
    }
  };
```

1　ずつ意　を確　します。

- `const handleSubmit = async (event) => { ... }` …「ログインボタンが押されたとき（フォーム　　）に　ばれる　　」を　　。`event`（イベント）には「何が　きたか」の　　が っています。

> 　　サーバーとの　　は「お　いして から　　が　るまで、　し　　がかかる」　　です。`await`（アウェイト：　つ）は「こ

> の　の　が　るまで、ここで　ってから　へ　む」という　。`await` を　う　に　は、　に `async`（　　：あとで　ってくる）を　ける　まりです。「　を　して、　がるのを　ってから　べる」イメージ。

- `event.preventDefault()` … フォームは　では「　するとページが　み　みされる」　をします。Reactでは　で　したいので、この　を　。これを　れると画が一　リロードされてしまいます。
- `setErrorMessage("")` …　回の　で　ていたエラー　を、いったん　します（§5.2の `errorMessage` state を　にする）。
- `try {` …「この　の　は　するかもしれない。　したら `catch`（　）へ　ぶ」という　。

> 「　するかもしれない　」を `try { }` で囲み、もし　（　外）が　きたら `catch { }` の　で　け　める　み。　は「サーバーが　ちている」「ネットが　れた」など　しうるので、　ずこれで　ります。　りの下に　る「安　ネット」のイメージ。

- `const response = await api.post("/api/login/", { employee_no: Number(employee_no) });` …　です。
  - `api.post(...)` … POSTメソッド（　。§2.2）で、`/api/login/` という　へリクエストを　る。
  - `{ employee_no: Number(employee_no) }` …　るデータ。`Number(...)` で　しています。

    > `<input type="number">` の　は、JavaScriptでは　は　（　：`"12345"`）として　きます。サーバー　は　の　員　を　しているため、`Number("12345")` → `12345` と　化してから　っています。

  - `await` が　いているので、サーバーの　が　るまでここで　ち、　を `response` に　れます。
- `const data = response.data;` …　の　から「　（JSONデータ）」を　り　して `data` へ。
- `if (data.status === "success")` …　の `status` が `"success"` か　。`===` は「完に　しいか」の　較（　まで一　を確　する　　較）。

- `navigate("/")` …　　なら、`"/"`（メインメニュー）へ画　を移　。`useNavigate`（§5.2　）で　意した　　を　っています。
- 　　以外なら、サーバーが　したメッセージ（`data.message`）を画　に　　。`||`（オア）は「　がなければ右を　う」。メッセージが　いときは「サーバーエラーが　　しました。」をします。
- `catch (err)` …　　そのものが　　したとき（サーバーがエラーを　した、ネットが　れた　）にここへ　ます。
- `axios.isAxiosError(err)` … そのエラーが「　　のものか」を　　。
- `if (err.response?.status === 401) navigate("/login")` … サーバーが　　を　したら、ログイン画　へ　します。

> 　　サーバーが「　　の　果がどうだったか」を3　の　　で　す　　ルール。　　なもの：
>
> - 　　…　　（OK）
> - 　　… リクエストが　　（Bad Request。　　ミスなど）
> - 　　…　　（Unauthorized。ログインが　　）
> - 　　…　　なし（Forbidden。ログインはしているが、この　　は　されない）
> - 　　…　つからない（Not Found）
> - 　　… サーバー　　エラー　アプリでは「401ならログイン画　へ　ばす」というがあり、サーバー　（§7）でも　ログイン　に401を　しています。フロントとバックで　　という　　が一　しているわけです。`?.`（オプショナルチェーン）は「`err.response` が　すれば `.status` を　る、　ければ何もしない（エラーにしない）」という安　な　き　です。

- 　　それ以外のエラーは、サーバーのメッセージ、または「　　なエラー」を　　します。

> 　　も　員　　も、「`try` で囲んで `api.post` / `api.get` し、`status` を　て　　、`catch` でエラー　　」という　を　り　します。ここを　解すれば、　の画　の　　も　めます。

画　の　た　は `03`　で　しく扱うので、ここでは　　だけを　います。

```
  // ① ここから画　の中　を　す
  return (
    // ② 画　全体を囲む
    <div className={styles["login-wrapper"]}>
      // ③ ロゴ画像
      <img src={logo} alt="業務工数システムロゴ" className={styles["login-logo"]} />
      // ④ 入力フォーム枠
      <form
        // ⑤　 信時に handleSubmit(§5.3)を呼ぶ ← ここが　！
        onSubmit={handleSubmit}
        // ⑥ キー入力時の処理(Enter　　信の　止)
        onKeyDown={(e) => {
          if (e.key === "Enter" && e.target instanceof HTMLInputElement &&
e.target.type !== "textarea") {
            //　　Enterでの　信を止め
            e.preventDefault();
            //　　入力欄からフォーカスを外す
            (e.target as HTMLInputElement).blur();
          }
        }}
      >
        // ⑦　 出し
        <h2>ログイン</h2>
        // ⑧ 入力欄をまとめる
        <div className={styles["search-bar"]}>
          // ⑨ ラベル
          <label htmlFor="numberInput">従業員番号</label>
          // ⑩ 入力欄
          <input
            //　　数値入力
            type="number"
            id="numberInput"
            //　　　　する値は state の employee_no(§5.2)
            value={employee_no}
            //　　入力が変わるたびに
            onChange={(e) => {
              const value = e.target.value;
              //　　state を更新(setNumber。§5.2)← ここが　！
              setNumber(value === "" ? "" : value);
            }}
            //　　入力必
            required
            className={styles["input-focus"]}
          />
          // ⑪ errorMessage(§5.2)があるときだけ
          {errorMessage && (
            //　　エラーを　いボックスで
            <div role="alert">{errorMessage}</div>
```

```
          )}

          // ⑫　信ボタン
          <button type="submit" className="blue_button">ログイン</button>
        </div>
      </form>
      // ⑬ ヘルプ画　へのリンク
      <Link to="/help" className={styles["help-button"]}></Link>
    </div>
  );
};

// ⑭ このLogin　品を、他から使えるよう公　する
export default Login;
```

の　　で　　なのは　の4　です。

- `onSubmit={handleSubmit}` … フォームが　　されると、§5.3で　　した `handleSubmit` が　ばれます。　　がここです。
- 　　　　　　　　　　　　　　　　　　　　… 　　の　　は state（`employee_no`）と　　し、　が　わるたびに `setNumber` で state を　　します。これにより「　　に　った　が、`handleSubmit` で　る　と　に一　」します。この「　　とstateを　びつける」やり　を　と　びます（`05`　で　　）。
- `{errorMessage && (...)}` … `errorMessage` が　でないとき（つまりエラーがあるとき）だけ、　いエラーボックスを　　。`handleSubmit` の　で `setErrorMessage(...)` した　　がここに　ます。
- `export default Login;` … この `Login`　　を「外　から　えるように　開」します。これにより `index.tsx`（§10）から `import Login` できるのです。

> 　　　　　　　　　　`import`（　り　せ）の　。「このファイルで　ったものを、　のファイルから　えるように外へ　開する」　　。`export default Login` は「このファイルの　　として `Login` を　開」という意　。§10で `index.tsx` がこれを `import Login from './MainPage/Login';` と　け　ります。

　　　　ユーザーが　　を　　（state　　）→ ボタンで `handleSubmit`　　→ `/api/login/` へPOST →　　で　　（　　なら画　移　、　　ならエラー　　）。　は、この `/api/login/` というリクエストが、サーバー　で　　　　　　　　を　ます。

`Login.tsx` が った `/api/login/` というリクエストは、Django で
に らして り けられます。その 応 が `kosu/urls.py`（ 106 ）です。

> 「 たリクエストのURLを て、
> の へ り けること」。 の が「 への ですね、おつなぎします」と
> 回すのに ています。Djangoでは、この 応 を `urls.py` というファイルに きます。

> URL 応 を くのに な と、各 の
> が かれたファイル （views）を り せます。

```
# ① URL定 の 具 path を取り寄せ
from django.urls import path
# ② ログイン の処理が入ったファイル
from .views import main_views
# ③ 工数入力の処理
from .views import kosu_views
# ④ 人員の処理
from .views import member_views
# ⑤ 班員の処理
from .views import team_views
# ⑥ 工数区分定 の処理
from .views import def_views
# ⑦ 問い合わせの処理
from .views import inquiry_views
# ⑧ バックアップ の 同期処理
from kosu.views import asynchronous_views
# ⑨ 定ファイル(settings.py)の値を む 具
from django.conf import settings
# ⑩ メディアファイル 信の 具
from django.conf.urls.static import static
```

- `from django.urls import path` … `path(...)`（パス）は「このURLは、この が 」という1 を くための です。

- `main_views`・`kosu_views` などは、　　ごとに　けられた　　。`from .views import ...` の `.`（ドット）は「このアプリ（`kosu`）の　の」という意　です。

> 　　Djangoで「リクエストを　け　って、　　して、レスポンスを　す」　割の　　。　アプリでは　　ごとにファイルを　けています（`main_views.py` ＝ログイン　、`kosu_views.py` ＝　　、…）。「　　の　で、　　ジャンルごとに　を　けている」イメージ。

`urlpatterns`（ユーアールエル・パターンズ）という　　のリスト（一　）に、URLと　　の　応を1　ずつ　べます。　　な　を　みましょう。

> 　　「このURLが　たら、この　　クラスを　ぶ」という　応を、　　の　だけ　べた一　です。

```
# ① URL対応　(リスト)のはじまり
urlpatterns = [
  # ② /login/ → main_views の Login が担当
  path('login/', main_views.Login.as_view(), name='login'),
  # ③ /logout/ → Logout が担当
  path('logout/', main_views.Logout.as_view(), name='logout'),
  # ④ /help/ → Help が担当
  path('help/', main_views.Help.as_view(), name='help'),
  # ⑤ /help_pass/ → HelpPass が担当
  path('help_pass/', main_views.HelpPass.as_view(), name='help_pass'),
  # ⑥ /main_menu/ → Menu が担当
  path('main_menu/', main_views.Menu.as_view(), name='main_menu'),
  # ⑦ 工数メニュー
  path('kosu_menu/', main_views.KosuMenu.as_view(), name='kosu_menu'),
  # ⑧ 工数の新　入力
  path('kosu_new/', kosu_views.KosuNew.as_view(), name='kosu_new'),
  # ...（中略　工数・定　・人員・班員・問い合わせ・　理　の各URLが　く）...
  # ⑨ ID付きURL(後　)
  path('kosu_update/<int:pk>/', kosu_views.KosuUpdate.as_view(), name='kosu_update'),
```

`path` の　　は3つの　　でできています。`path('login/', main_views.Login.as_view(), name='login')` を　解すると：

1. [illegible] … URLの [illegible] （パターン）。`/api/login/` の `api/` を [illegible] いた [illegible] 。
2. [illegible] … このURLが [illegible] たときに [illegible] ぶ [illegible] 。「`main_views` ファイルの の `Login` クラス」を [illegible] に [illegible] 。`.as_view()` は「クラスを、URLから [illegible] べる [illegible] に [illegible] する」おまじない。
3. [illegible] … この [illegible] 応に [illegible] ける「あだ [illegible] 」。コードの [illegible] の [illegible] からURLを [illegible] するのに [illegible] えます。

> [illegible] `Login.tsx`（§5.3 ）が `api.post("/api/login/", ...)` を [illegible] ると、Djangoは `/api/` の [illegible] ろの `login/` を [illegible] て、この `urls.py` の [illegible] にたどり [illegible] きます。そして [illegible] の `main_views.Login` を [illegible] び [illegible] します。
> [illegible] のが [illegible] えますか？これがWebアプリの [illegible] です。

の `path('kosu_update/<int:pk>/', ...)` の `<int:pk>` は「ここに [illegible] （int）が [illegible] る。それを `pk` という [illegible] で [illegible] に [illegible] す」という意 [illegible] です。たとえば `/api/kosu_update/42/` なら、`pk=42` として [illegible] に [illegible] きます。

> [illegible] Primary Key の [illegible] 。データベースの各 [illegible] に [illegible] で [illegible] く「 [illegible] しない [illegible] （ID）」。`/kosu_update/42/` は「ID [illegible] 42の [illegible] データを [illegible] する」という意 [illegible] です。§8の `models.py` で、各 [illegible] に [illegible] でこのID（ [illegible] ）が [illegible] られる [illegible] みを [illegible] ます。

```
# 工数のバックアップ
path('kosu_backup/', asynchronous_views.backup, name='kosu_backup'),
# 工数の み み(復元)
path('kosu_load/', asynchronous_views.backup, name='kosu_load'),
# 定 のバックアップ
path('def_backup/', asynchronous_views.backup, name='def_backup'),
# ... 同様に member / team / inquiry / setting などのバックアップ・復元が並ぶ ...
```

ここは「いろいろなバックアップ・ URL」が、 [illegible] している のが [illegible] です。どのデータを扱うかは、URL やリクエストの [illegible] で [illegible] けます（ [illegible] は [illegible] のバックアップ [illegible] で扱います）。`.as_view()` が [illegible] いていないのは、これがクラスではなく [illegible] だからです（ [illegible] ビューはそのまま [illegible] せます）。



```
# ① 日付・時刻を扱う　具
import datetime
# ② OS(ファイルパス　)を扱う　具
import os
# ③ Pythonの実　環境を扱う　具
import sys
# ④ ログ(動作　　)を残す　具
import logging
# ⑤ .env(　密　定)を　む　具
import environ
# ⑥ JSONを扱う　具
import json
# ⑦ ファイルの場所を扱う　具
from pathlib import Path
# ⑧ HTMLを　す　具
from django.shortcuts import render
# ⑨ JSONで　事を　す　具
from django.http import JsonResponse
# ⑩ ビューの基本クラス
from django.views import View
# ⑪　 定値を　む　具
from django.conf import settings
# ⑫ API用のビュー基本クラス(よく使う)
from rest_framework.views import APIView
# ⑬ API用の　事を　す　具(よく使う)
from rest_framework.response import Response
# ⑭ ステータスコードの名前付き定数
from rest_framework import status
# ⑮ DBのモデル　を取り寄せ
from ..models import member, Business_Time_graph, kosu_division, team_member,
administrator_data, AsyncTask, History
# ⑯ データ変換器
from .serializers import MemberSerializer, AdministratorSerializer, KosuSerializer,
DefSerializer, TaskSerializer, HistorySerializer
# ⑰ ページ分割　の　助　具
from ..utils.main_utils import CustomPagination, get_all_model_names_in_myapp
```

に　　なのは　の3つです。

- 　　… 「APIのビューはこれを　　に　る」という　　クラス。　アプリのほぼ　ビューがこれを　　しています。
- 　　… フロントへJSONで　　を　す　　。`Login.tsx`（§5.3　）が　け　る`response.data`の　　は、これで　られます。
- 　　… データベースの　（モデル）を　り　せ。`..`は「2つのフォルダ（`kosu/`）」の意　。`member`（　員　）を　って「この　　員　　は　　されている

か」を　べます。

> Djangoで「データベースの　（テーブル）」をPythonのクラスとして　したもの。1つのモデル＝1つの　。`member` モデルは「　員　」、`Business_Time_graph` モデルは「　　の　」です。§8で `models.py` を　み、これらの　　を確　します。

> データベースから　り　した「Pythonのオブジェクト」を、フロントへ　れる「JSON」に　　する　　。　に、フロントから　たJSONをDBに　　できる　に　す　　もします。`MemberSerializer` は「　員データ　JSON」の　　です。

いよいよ核　です。`Login.tsx` がPOSTしたリクエストを、この `Login` クラスの `post` メソッドが　け　ります。

> フロントから　られてきた　　員　　を　り　し、(1) JSONとしてきさこと　さトか確　　、よよ オ　トに　トするル

```
# ログイン
# ① ログイン処理(APIView を土台に)
class Login(APIView):
  # POST処理
  # ② POSTリクエストが来たときの処理
  def post(self, request):
    # ③ 失敗しうる処理の　始
    try:
      #　 られてきたデータ確
      # ④ リクエスト本文をJSONとして　み　む
      data = json.loads(request.body)
      # ⑤ その中から従業員番号を取り出す
      input_number = data.get('employee_no')
    # ⑥ JSONとして　めなかったら
    except json.JSONDecodeError:
      # ⑦ 400エラーを　す
      return JsonResponse({'status': 'error', 'message': 'JSON形式が正しくありません。'}, status=status.HTTP_400_BAD_REQUEST)

    # POSTされた従番が人員データにある場合、セッションに保存&最新工数区分定　取得
    # ⑧ その番号が member　にあるか?
    if member.objects.filter(employee_no=input_number).exists():
      # ⑨ あれば「ログイン中の番号」を　える(セッション)
      request.session['login_No'] = input_number
      # ⑩ 工数区分定　の最新版を取得
      def_Ver = kosu_division.objects.order_by("id").last()

      # 取得した工数区分定　をセッションに保存
      # ⑪ もし定　が1件も無ければ
      if not def_Ver:
        # ⑫ エラー
        return JsonResponse({'status': 'error', 'message': '利用可　な工数区分がありません。ERROR052'}, status=status.HTTP_400_BAD_REQUEST)
      # ⑬ 定　があれば
      else:
        # ⑭ その定　名を　える(セッション)
        request.session['input_def'] = def_Ver.kosu_name
        # ⑮ 「成功」を　す ← Login.tsx の⑦へ!
        return JsonResponse({'status': 'success'})

    # ⑯ 番号が member　に無ければ
    else:
      # ⑰ エラー
      return JsonResponse({'status': 'error', 'message': '入力された従業員番号は登　がありません。ERROR048'}, status=status.HTTP_400_BAD_REQUEST)
```

1　ずつ　みます。

- `class Login(APIView):` … 「`Login` というAPIのビューを　る」。括　の　の `APIView` は「これを　　にする（　　する）」という意　。

> 　　は「ある　　の　　・データを　ひとまとめにした　　」。　　は「　　のクラスの　　を引き　いで、　　のクラスを　る」こと。`Login(APIView)` は「`APIView` が　つ　　（リクエストの　け　り　など）を引き　いだ　で、ログイン　　の　　を　す」という意　です。　　の「　　のレシピを　　に、　　のアレンジを　す」イメージ。

- `def post(self, request):` … 「POSTリクエストが　たら、この　　を　　」。`request`（リクエスト）に、フロントから　られた　　すべてが　っています。`def` は「　　（　　のまとまり）を　　する」Pythonの　　です。

> 　　§2.2のHTTPメソッドと　応しています。`Login.tsx` が `api.post(...)`（POST）で　ったので、サーバー　も `def post` が　応します。もし　GETで　たら `def get` が　ばれます。　　しているのが　DRF（　）のお　です。

- 　　`try / except` … `json.loads(request.body)` でリクエスト　　をJSONとして　み　みます。`data.get('employee_no')` で、その　から　員　　を　り　します。もし　られた　がJSONとして壊れていたら（`JSONDecodeError`）、400エラー（　　なリクエスト）を　します。

> 　　これは `Login.tsx`（§5.3　）が `{ employee_no: Number(employee_no) }` という　　で　ったデータと　　しています。フロントが `employee_no` という　　で　り、バックが `employee_no` という　　で　け　る――この　　が、フロントとバックの「　」です。どちらかが　　を　えると　　が壊れます。

- `if member.objects.filter(employee_no=input_number).exists():` …　　です。ここが§4の　　で「　データベースに　い　わせ」と　いた　。
  - `member.objects` … `member`　（　員　。§8で　　を　ます）の　データ。
  - `.filter(employee_no=input_number)` …「　　員　　が、　　された　　と一　する　だけ」に　り　み。
  - `.exists()` …「そういう　が1つでもあるか？（True / False）」を　す。

> つまりこの1　は「　　」をデータベースに　ねています。これがログイン　　の　　です。　アプリのログインはパスワードではなく「　　が　　されているか」で　　する　　です。

- `request.session['login_No'] = input_number` …　　があれば、　　に「ログインの　　員　　」を　　します。

> 「この　はログイン　みだ」という　　を、サーバーが一　　覚えておく　　み。ログインのたびにブラウザに　さな　　（クッキー）を　し、　回以　のリクエストでその　　を　て「あ、さっきログインした　だ」とかるようにします。これにより、画　を移　しても　　のです。§7.4で、　のメニュー画　がこのセッションを　って「ログイン　みか」を確　しているのを　ます。

- `def_Ver = kosu_division.objects.order_by("id").last()` …　　（　　の一　。§8の `kosu_division`）の　から、　　を　り　します。`.order_by("id")` で　　に　べ、`.last()` で　　の1　を　　。
- 　もし　　が1　も　ければ（`not def_Ver`）、`ERROR052` という　　きエラーを　します。

> ユーザーが「エラーが　た」と　ってきたとき、　　があれば「どこで　きたエラーか」をコードから　　に　　できます。　　をラクにする　　です。

- `request.session['input_def'] = def_Ver.kosu_name` …　　の　　もセッションに　。以　の　　で「どの　　を　うか」の　　になります。
- `return JsonResponse({'status': 'success'})` …　　この `{'status': 'success'}` というJSONが、ネットワークを　って `Login.tsx`（§5.3　）の `data.status === "success"` で　け　られ、`navigate("/")` でメインメニューへ　みます。　　のです。
- 　　が　　されていなければ、`ERROR048`「　　された　　員　　は　　がありません」を400エラーで　します。これは `Login.tsx`（§5.3　）の `catch`　で　け　られ、エラーメッセージとして画　に　　されます。

> `Login.tsx`（　　・　）→ `urls.py`（`/login/` を `Login` へ　り　け）→ `main_views.py`（`member`　に　い　わせ・　/　を　）→ JSONで　→ `Login.tsx`（　なら画　移）。
>
> ―― この　格が、　アプリのすべての　に　します。

`Login` だけでなく、`main_views.py` には　くのクラスがあります。　として `Help`（ヘルプ画の　化　）を　く　ておきます。これは「データを　（　荷　）に　す」　です。

> ヘルプ画　から「　員データを　化」「　を　化」「　を　化」のいずれかが　されたら、その　を　して、　のデフォルトデータ（　1　・　1つ・　1つ）を　り　します。

```
class Help(APIView):
  def post(self, request):
    # ①   られてきたデータ
    post_data = request.data
    # ② 人員初期化の   否
    memberReset = post_data.get('memberReset')
    # ③ 定   初期化の   否
    defReset = post_data.get('defReset')
    # ④   定初期化の   否
    settingReset = post_data.get('settingReset')

    try:
      # ⑤ 人員初期化が   求されたら
      if memberReset:
        # ⑥ 人員   を全削
        member.objects.all().delete()
        # ⑦ デフォルト   理   (番号12345)を作成
        default_member = member(employee_no=12345,
                                name='admin',
                                shop='その他',
                                #    権   あり
                                authority=True,
                                #      理
                                administrator=True,
                                #    各   休憩時   の初期値(後略)
                                break_time1='#11401240',
                                # ...(休憩時   の   定が   く)...
                                )
        # ⑧ データベースに保存
        default_member.save()
      # ...(defReset / settingReset も同様にデフォルトを作り直す)...
      # ⑨ 成功
      return Response({'status': 'success', 'message': 'OK'},
status=status.HTTP_200_OK)
    # ⑩ 何か失敗したら
    except Exception as e:
      return Response({'status': 'error', 'message': 'データの初期化中にエラーが発生しました。'}, status=status.HTTP_400_BAD_REQUEST)
```

    :

- … `employee_no=12345`、`authority=True`（   ）、`administrator=True`（   ）で   られます。
  ということです。これは引き   ぎで   に   な   です。

- [illegible] … 「 す」。⚠️ な です。 データに して安易に して はいけません。

> ⚠️ `Help` クラスの 化 は な です。 として、これがどういう か（ にデフォルト へ すためのもの）を 解しておき、 よう をつけてください。

`main_views.py` の各メニュー画 （`Menu`・`MemberMenu` など）には、 が り し れます。`Menu`（メインメニュー）を に ます。

> メインメニュー画 を開くとき、(1) セッションに「ログイン の 」があるか確 し、(2) ければ401エラー（ログインへ れ）を し、(3) あればその 員データと データを り してフロントへ します。

```
class Menu(APIView):
  # ① GET(画　　)のとき
  def get(self, request):
    # ② セッションからログイン番号を取り出す
    login_no = request.session.get('login_No')
    # ③ セッションから定　名を取り出す
    def_ver = request.session.get('input_def')

    # ④ ログイン番号が無ければ(未ログイン)
    if not login_no:
      # ⑤ 401
      return Response({'status': 'error', 'message': 'ログイン情報が確　できません。'}, status=status.HTTP_401_UNAUTHORIZED)
    # ⑥ 定　名も無ければ
    if not def_ver:
      return Response({'status': 'error', 'message': '使用する工数区分定　情報が確　できません。'}, status=status.HTTP_401_UNAUTHORIZED)

    try:
      # ⑦ その番号の人員データを取得
      member_data = member.objects.get(employee_no=login_no)
    # ⑧ もし　つからなければ
    except member.DoesNotExist:
      return Response({'status': 'error', 'message': 'ユーザー情報が　つかりません。'}, status=status.HTTP_401_UNAUTHORIZED)
    # ...　定データ取得、シリアライザでJSON化して　す ...
```

ここで `Login.tsx`（§5.3　）の「401ならログイン画　へ」が　いてきます。`Menu` 画　を開いたとき、　　、フロントはそれを　て　　にログイン画　へ　します。

されているのです。

> 　　`main_views.py` の各メニュークラスは、この「セッション確　→　ければ401 → あれば　員データ　　→　　チェック → データをJSONで　す」という　　をしています。1つ　解すれば、　りはほぼ　じパターンの　り　しです。`MemberMenu`（258　　）では、加えて `if not member_data.authority:`（　　が　ければ403）という　　が　っているのが違いです。

---

§7で `member.objects.filter(...)` のように、データベースの　に　い　わせていました。その　　が `kosu/models.py` です。「どんなデータを、どんな　で　　するか」を　　しています。

> （　　ソフトでたとえる）
>
> - 　　… データを　　する「　　」　　。Excelブック1　のイメージ。
> - 　　…　　の　の1つの「　（シート）」。　アプリの `member`（　員）・`Business_Time_graph`（　　）などが、それぞれ1　の　。
> - 　　…　の1　。「　　員1　ぶんの　　」「　　1　」など。
> - 　　…　の1　。「　　員　　」「　　」など、データの　　。Djangoでは、　　、　　に　応します。

```
# ① Djangoのモデル機　を取り寄せ
from django.db import models
# ② JSON型のカラムを使う　具(Historyで使用)
import jsonfield
```

ログイン　　で　われた、　　の　です。

> 　　員1　ぶんの　　（　員　・　・　ショップ・　・　フラグ・各　　ごとの　　など）を　する「　員　」の　です。

```
# ① member  の    図(models.Model を  承)
class member(models.Model):
  # ② 所属ショップの  択  一
  shop_list = [
    #   (左  保存される値, 右  画      )
    ('P', 'P'),
    ('R', 'R'),
    ('W1', 'W1'),
    # ... (W2, T1, T2, A1, A2, J など) ...
    ('その他', 'その他'),
    ('    以上(P,R,T,その他)', '    以上(P,R,T,その他)'),
    ('    以上(W,A)', '    以上(W,A)'),
    #    異動・    した人の区分
    ('異動・    ', '異動・    '),
  ]

  # ③ 従業員番号(整数)← ログイン    で使う  !
  employee_no = models.IntegerField('従業員番号')
  # ④ 氏名(最大100文字)
  name = models.CharField('氏名', max_length=100)
  # ⑤ 所属(②の  択  から  ぶ)
  shop = models.CharField('ショップ', choices=shop_list, max_length=15)
  # ⑥ 権  あり/なし(真偽値)← 班員機  などの可否
  authority = models.BooleanField('権  ')
  # ⑦  理  か(真偽値)←  理  機  の可否
  administrator = models.BooleanField('  理  ')
  # ⑧ 1直(勤務形態)の昼休憩時
  break_time1 = models.CharField('1直昼休憩時  ', max_length=9)
  # ⑨ 1直の残業休憩1
  break_time1_over1 = models.CharField('1直残業休憩時  1', max_length=9)
  # ... (break_time1〜6、各 over1〜3 まで、勤務形態×休憩区分の  み合わせが  く) ...
  # 休憩エラー判定の有効/無効
  break_check = models.BooleanField('休憩エラー有効チェック', null=True)

  # ⑩ この  を文字で  すときの  現
  def __str__(self):
    #   氏名を  す(  理画  などで  やすく)
    return self.name
```

1 ずつ、カラム の み を学びます。

- `employee_no = models.IntegerField('従業員番号')` …「`employee_no` という の、(Integer) の 。 は『 員 』」。 です。

> カラムの「データの　」を　するもの。`IntegerField` ＝　、`CharField` ＝　い　、`TextField` ＝　い　、`BooleanField` ＝True/False、`DateField` ＝　、`DateTimeField` ＝　。　を　めることで、データベースが「この　には　しか　らない」などと　してくれます。

- `name = models.CharField('氏名', max_length=100)` …　の　。`max_length=100` は「　100　」。`CharField` は　ず　を　します。
- `shop = models.CharField('ショップ', choices=shop_list, max_length=15)` … `choices=shop_list`（　の一　）を　すると、　になります。画　ではドロップダウンになります。
- `authority`・`administrator` … `BooleanField`（　＝True/False）。§1.3で　した「3　階の　」を、この2つの　（フラグ）で　しています。§7.4で `if not member_data.authority:` のように　われていました。
- `break_time1` ～ …　（1　・2　・3　・　・　1　・　2　の6　）ごとに、　＋　3つ、　24　の　カラムが　びます。`max_length=9` で、`#11401240` のような　（`#` ＋開　4　＋　4　）で　を　します。

> では　シフト（　）ごとに　が違うためです。　を　しく　するには「いつ　で、いつ　いていたか」が　なので、　ごと・　ごとに　を　たせています。

- `def __str__(self): return self.name` … この　（レコード）を　で　すときの　た　。Django　画　などで「ID　」ではなく「　」で　され、　に　かりやすくなります。

> モデル　でよく　る　。`null=True` は「データベース　で　（　）を　す」、`blank=True` は「　フォーム　で　を　す」。`pop_up1 = models.CharField(..., null=True, blank=True)` のように、　でない　に　けます。

> member モデルには `id` の　が　たりませんが、Djangoは　。これが §6.2 の `/kosu_update/<int:pk>/` で　われる　です。`settings.py`（§12）の `DEFAULT_AUTO_FIELD` がこの　IDの　を　めています。

の　　データです。「　が・いつ・何の　　を・何　　したか」を1　ずつ　　します。

```
# ① 工数　　の
class Business_Time_graph(models.Model):
  # ②　　した人の従業員番号
  employee_no3 = models.IntegerField('従業員番号')
  # ③ 人員　への参照
  name = models.ForeignKey(member, verbose_name='氏名', null=True,
on_delete=models.SET_NULL)
  # ④ 使った定　のVer
  def_ver2 = models.CharField('工数区分定　Ver', max_length=100, blank=True,
null=True)
  # ⑤ 就業日(日付)
  work_day2 = models.DateField('就業日')
  # ⑥ 勤務シフト(直)
  tyoku2 = models.CharField('直', max_length=2, blank=True, null=True)
  # ⑦ 作業内容(コード化)
  time_work = models.CharField('作業内容', max_length=288, blank=True, null=True)
  # ⑧ 作業の
  detail_work = models.CharField('作業　　', max_length=32676, blank=True, null=True)
  # ⑨ 残業時
  over_time = models.IntegerField('残業時　', blank=True, null=True)
  # ⑩ 昼休憩時
  breaktime = models.CharField('昼休憩時　', max_length=9, blank=True, null=True)
  # ... (残業休憩1〜3、就業形態など) ...
  # ⑪ 入力が完了/正常か(OK/NG)
  judgement = models.BooleanField('工数入力OK_NG', null=True)
  # ⑫ 休憩を変更したか
  break_change = models.BooleanField('休憩変更チェック', null=True)

  def __str__(self):
    return str(self.id) + '__' + str(self.work_day2) + ':' + str(self.employee_no3)
```

- `name = models.ForeignKey(member, ...)` …　　です。

> 「　の　のレコードを　し　す」カラム。ここでは　　から `member`（　員　）の　　の　を　しています。「この　　は、　員の◯◯さんのもの」という　　を　ります。Excelで「　シートの　を　する」イメージ。 `on_delete=models.SET_NULL` は「もし　　の　員が　　されたら、

> このカラムを （NULL）にする」という意 。 員を しても は えず、
> けだけ外れます。

- `judgement`（ OK_NG）… この が「ちゃんと 完 したか」の 。 員フォロー （`TeamMenu`）では、この `judgement` が `False` なら「 があります」としています。

§7.2 で「 を 」していた、 の です。このモデルは ので、だけ 解します。

```
# ① 工数区分定 の
class kosu_division(models.Model):
  # ② 定 のバージョン名
  kosu_name = models.CharField('工数区分定 Ver名', blank=True, null=True,
max_length=100)
  # ③ 区分1の名前
  kosu_title_1 = models.CharField('工数区分名1', blank=True, null=True,
max_length=100)
  # ④ 区分1の定
  kosu_division_1_1 = models.TextField('定 1', blank=True, null=True)
  # ⑤ 区分1の作業内容
  kosu_division_2_1 = models.TextField('作業内容1', blank=True, null=True)
  # ⑥ 区分1が変動か固定か
  kosu_division_3_1 = models.BooleanField('変動/固定1', null=True)
  # ... kosu_title_2 ～ kosu_title_50、それぞれ _1～_3 が く 最大50区分 ...
  kosu_title_50 = models.CharField('工数区分名50', blank=True, null=True,
max_length=100)
  kosu_division_1_50 = models.TextField('定 50', blank=True, null=True)
  kosu_division_2_50 = models.TextField('作業内容50', blank=True, null=True)
  kosu_division_3_50 = models.BooleanField('変動/固定50', null=True)

  def __str__(self):
    return str(self.id) + ' : ' + str(self.kosu_name)
```

ポイントは「 」ことです。各 は「 （`kosu_title_n`）・ （`kosu_division_1_n`）・（`kosu_division_2_n`）・ / （`kosu_division_3_n`）」の4つで されます。

> 、データベースの 科 には「 」を の に けて、1レコード1 にするのが です。しかし アプリは「1つの Ver＝1レコードに50 を横 び」という を っています。これは扱いがシンプルになる 、 が になります。
> ことが です（ いているものには があります）。§7.2 の `kosu_division.objects.order_by("id").last()` が、この の「 の1レコード（＝ の ― ）」を り していたわけです。

りのモデルも、 割だけ押さえておきましょう。

`team_member`（91 〜）	（チーム）の 。15 を	`member1`〜`member15`、`follow`（フォローON/OFF）
`def_choice`（323 〜）	の（ と のペア）	`def_symbol`（A〜Zなどの ）、`def_select`
`administrator_data`（334 〜）	（― ・ い わせなど）	`menu_row`（― ）、`administrator_employee_no1`〜3
`inquiry_data`（355 〜）	い わせ（ ・ ・ い わせ）	`content_choice`、`inquiry`（ ）、`answer`（回 ）
`AsyncTask`（375 〜）	タスク（バックアップ ）の	`task_id`、`status`、`MAX_RECORDS=1000`（ い は ）
`Operation_history`（405 〜）		`post_page`、`operation_models`、`status`
`History`（419 〜）	データ の ログ	`operation`、`changes`（JSON ）、`MAX_RECORDS=500000`

`AsyncTask` と `History` には `save()` メソッドが きされており、「レコードが（`MAX_RECORDS`）を えたら、 いものから する」 みが っています。これにより、ログが に まってデータベースが するのを いでいます。

```
class AsyncTask(models.Model):
  # 保持する最大件数
  MAX_RECORDS = 1000

  # タスクの一意なID
  task_id = models.CharField(max_length=255, unique=True)
  # 状態(pending/success/error)
  status = models.CharField(max_length=50, choices=[
    ('pending', 'Pending'),
    ('success', 'Success'),
    ('error', 'Error')
  ])
  #   果
  result = models.TextField(null=True, blank=True)
  # 作成日時(  動)
  created_at = models.DateTimeField(auto_now_add=True)
  # 更新日時(  動)
  updated_at = models.DateTimeField(auto_now=True)

  # 保存時に上  を  えたら古い  に削
  def save(self, *args, **kwargs):
    super().save(*args, **kwargs)
    current_count = self.__class__.objects.count()
    if current_count > self.MAX_RECORDS:
      excess_count = current_count - self.MAX_RECORDS
      oldest_records = self.__class__.objects.order_by('created_at')[:excess_count]
      for record in oldest_records:
        record.delete()
```

ここは です。`member`（ 員）でログインし、`Business_Time_graph`（ ）に を め、`kosu_division`（ ）で を します。

のが の になります。

---

§6で た `kosu/urls.py` は「`/api/login/` のうち `login/` の 」を していました。では「`/api/` という 」 は、どこで まっているのでしょうか。それが、プロジェクト の

`hozen_another/urls.py`（ 18 ）です。

> Djangoでは、 が「サイト 」、 が「その の のまとまり」です。 アプリでは `hozen_another/` がプロジェクト（ ・ 位URL）、`kosu/` がアプリ（ の ）。`hozen_another/urls.py` が「 の 」、`kosu/urls.py` が「各 屋の の案 」という階 です。

> ブラウザから たURLを、その で きく り けます。`/admin/` はDjango 画 へ、`/api/` で まるものは `kosu` アプリのURL（§6）へ、それ以外は Reactの画 へ、という3つの きな です。

```
# ① Django標準の　理画
from django.contrib import admin
# ② URL定　の　具(include=他のurls.pyを取り　む)
from django.urls import path, include, re_path
# ③ React　　処理とmanifest処理を取り寄せ
from kosu.views.main_views import react_view, manifest
# ④　定値を　む
from django.conf import settings
# ⑤　的ファイル　信の　具
from django.conf.urls.static import static



# ⑥ 最上位のURL対応
urlpatterns = [
  # ⑦ /admin/ → Django　理画
  path('admin/', admin.site.urls),
  # ⑧ /BA で始まるURL → kosu/urls.py へ
  path('BA', include('kosu.urls')),
  # ⑨ /api/ で始まるURL → kosu/urls.py へ ←　　!
  path('api/', include('kosu.urls')),
  # ⑩ /manifest.json → アプリ情報を　す
  path('manifest.json', manifest),
  # ⑪ admin以外の全URL → Reactの画　を　す
  re_path(r'^(?!admin).*$', react_view),
]

# ⑫　発モードのときだけ
if settings.DEBUG:
    # ⑬　的ファイル　信を　加
    urlpatterns += static(settings.STATIC_URL, document_root=settings.STATIC_ROOT)
```

1　ずつ　みます。

- `include`（インクルード）…「　の `urls.py` を、ここに　り　む」　　。これで階　　な URL　　ができます。
- `path('admin/', admin.site.urls)` … `/admin/` で　まるURLは、Django　　の　　へ。これはDjangoが　意してくれる、データベースを　　たり　　したりできる　　な　画　です。
- `path('api/', include('kosu.urls'))` …　　です。`/api/` で　まる URLが　たら、　ろの　　を `kosu/urls.py`（§6）に　して、そこで　　を　めてもらいます。

> つまり `/api/login/` は、 、2 階のルーティングを っていたのです。`/BA` でも じ `kosu.urls` を えるようにして あります（ でのアクセス ）。

- `path('manifest.json', manifest)` … `/manifest.json` はアプリ （アプリ ・アイコンなど）を す へ。スマホでホーム画 に 加したときの などに われます。
- `re_path(r'^(?!admin).*$', react_view)` … です。
  - `re_path` … （パターンマッチ）でURLを する 。
  - `r'^(?!admin).*$'` … 「`admin` で あらゆるURL」というパターン。
  - `react_view` … そのURLに して、Reactのページ（`index.html`）を す 。

  > アプリはフロントとバックが しているので、`/` ・`/kosu-menu` ・`/member-list` など 。ユーザーがどの画 URLを開いても（adminとapi以外なら）、サーバーは じReactの（`index.html`）を し、 のです。これを とびます。「1 のHTML を に み み、その は画 の だけをJavaScriptで し える」 のWebアプリ。ページ を み み さないので、画 移が くスムーズです。 アプリはこの です。

- `if settings.DEBUG:` … `settings.py` の `DEBUG` が `True`（開 モード）のときだけ、ファイル（CSS・画 など）の を 加します。 では の （WhiteNoise。§12）で するため、ここは開 です。

> これは です。`/admin/` → 画 、`/api/` →バックエンド （kosu/urls.py ）、それ以外→Reactの画 、という3 。 されています。

---

§9の　で「画　　URLはReactに　せる」と　かりました。では、Reactは「どのURLで、どの画　　を　すか」を　　のか。それが、フロントエンドの `frontend/src/index.tsx`（　180　）です。

> 　　フロント　で「URLに応じて　　する画　　を　り　える」　　。　アプリは `react-router-dom`（リアクト・ルーター・ドム）というライブラリを　います。§9　でサーバーが「画　はReactに　せた」と　った、その　　を　　に下すのがこのルーターです。

> 　　Reactとルーターの　　、　　CSS、そしてアプリの　画　　（ログイン・各　メニュー・一　・　　など　50　）を　り　せます。

```
// ① React本体
import React from 'react';
// ② Reactを画　に描く　具
import ReactDOM from 'react-dom/client';
// ③ ルーターの　具一式
import { BrowserRouter as Router, Routes, Route, useLocation } from "react-router-dom";
// ④ 全画　共　のCSS(03　で　　)
import './styles/global.css';
// ⑤ §5で　んだログイン画　を取り寄せ
import Login from './MainPage/Login';
// ⑥ ヘルプ画
import Help from './MainPage/Help';
// ⑦ メインメニュー画
import MainMenu from './MainPage/MainMenu';
// ⑧ 工数メニュー
import KosuMenu from './KosuPage/KosuMenu';
// ⑨ 工数入力
import KosuNew from './KosuPage/KosuNew';
// ... (KosuList, KosuEdit, ... 工数・定　・人員・班員・問い合わせ・　理　の全画　が
く) ...
// 最後　操作履歴
import AdministratorHistoryDetail from
'./AdministratorPage/AdministratorHistoryDetail';
```

- 　ルーターの4つの　　を　り　せます。
  - `BrowserRouter`（`as Router` で `Router` と改　）… ルーター　　を　　にする外　。
  - `Routes`・`Route` …「このURLなら、この　　」という　応を　くための　　。
  - `useLocation`（ユーズロケーション）…「　どのURLにいるか」を　る　　。
- `import Login from './MainPage/Login';` …　　　　　　　　　　をここで　り　せています。`export` と `import` が、ファイルをまたいでつながっているのが　かります。
- 　　50　の画　　をすべて　り　せます。フォルダ　（`MainPage`・`KosuPage`・`MemberPage` …）が、§1.2の　　グループと　応しているのに　　してください。

```
// ① アプリ全体を　す　品
const App: React.FC = () => {
  // ② 今いるURLの情報を取得
  const location = useLocation();

  // ③ URLが変わるたびに実　する処理
  React.useEffect(() => {
    // ④ URLごとのページタイトルー
    const routeTitles: { [key: string]: string } = {
      //    /login のときのタブ　　名
      "/login": "ログイン - 業務工数システム",
      "/help": "ヘルプ - 業務工数システム",
      "/": "Main Menu - 業務工数システム",
      "/kosu-menu": "工数Menu - 業務工数システム",
      // ... (全URLに対応する日本　タイトルが　く) ...
      "/manager-history-detail/:id": "データ操作履歴　　- 業務工数システム",
    };

    // ⑤ ブラウザのタブ名を　定
    document.title = routeTitles[location.pathname] || "業務工数システム";
  // ⑥ location(URL)が変わったら③を再実
  }, [location]);
```

- 　　… 「location（URL）が　わるたびに、　の　を　　する」　　み。

> 　　「画　が　　された　や、　　の　が　わった　に、何か　　をする」ためのReactの　　。[location] の　　が「これが　わったら　　」という　　。ここでは「URLが　わったら、ブラウザのタブのタイトルを　える」のに　っています。05　で　　。

- 　　URLごとに、ブラウザのタブに　すタイトル（　　:「ログイン - 　　システム」）を　めています。document.title = ... でタブ　を　　。|| は「一　に　いURLなら『　　システム』を　　」という　　。

いて、　　である　　です。

> 　　「このURLパスのときは、この画　　を　　する」という　応を、　画　ぶん　べます。§9　でサーバーから　されたReactが、ここを　て

> の画　を　めます。

```
  return (
    // ① ルート対応のまとまり
    <Routes>
      // ② /login → Login画　(§5)を
      <Route path="/login" element={<Login />} />
      // ③ /help → Help画
      <Route path="/help" element={<Help />} />
      // ④ / → メインメニュー(ログイン成功後の　き先!)
      <Route path="/" element={<MainMenu />} />
      // ⑤ /kosu-menu → 工数メニュー
      <Route path="/kosu-menu" element={<KosuMenu />} />
      // ⑥ /kosu-new → 工数入力
      <Route path="/kosu-new" element={<KosuNew />} />
      // ⑦ /kosu-list → 工数一
      <Route path="/kosu-list" element={<KosuList />} />
      // ⑧ /kosu-update/42 など → 工数　　(:idは可変)
      <Route path="/kosu-update/:id" element={<KosuEdit />} />
      // ...(定　・人員・班員・問い合わせ・　理　の全ルートが　く)...
      // 最後
      <Route path="/manager-history-detail/:id" element={<AdministratorHistoryDetail
/>} />
    </Routes>
  );
};
```

- `<Route path="/login" element={<Login />} />` …「URLが `/login` のとき、`Login` 画（§5）を　　する」。`element={<Login />}` が「　　する　　」です。
- `<Route path="/" element={<MainMenu />} />` … `Login.tsx` が　　に `navigate("/")` で `/` へ移　すると、ルーターがこの　を　て `MainMenu`（メインメニュー）画　を　　します。
  　のです。
- `path="/kosu-update/:id"` の `:id` …「ここは可　（どんな　でも　る)」を　し、`/kosu-update/42` のようなURLにマッチします。§6.2　でサーバー　の `<int:pk>` を　ましたが、
  　わけです。

った App を、index.html の の root という に に き みます。これがアプリ の の一 です。

```
// ① index.html内の#rootを取得
const root = ReactDOM.createRoot(document.getElementById('root') as HTMLElement);

// ② そこにReactアプリを描く
root.render(
  // ③  発時の問  検出を有効にする外枠
  <React.StrictMode>
    // ④ ルーターを有効化(URL切替を可  に)
    <Router>
      // ⑤ §10.2で作った App を描画
      <App />
    </Router>
  </React.StrictMode>
);
```

- document.getElementById('root') … index.html の にある をします。 です。§9 でサーバーが した index.html の の、この root に、Reactが画 を し みます。
- <Router> … これで めてルーティング（URLに応じた画 ）が になります。<App /> をこれで むことで、§10.2のルート が くようになります。
- <React.StrictMode> … 開 に な を つけやすくする「 格モード」。 には影 しません。

これは です。 画 を り せ、URLと画 の 応 （Routes / Route ）を し、 に index.html の root にアプリを き みます。 のが になります。

Login.tsx （ → ）→ hozen_another/urls.py （/api/ →kosu）→ kosu/urls.py （login/ →Login）→ main_views.py （DB → ）→ JSON → Login.tsx （navigate("/") ）→

> `index.tsx`（`/` → `MainMenu`　　）。
> のを　コードで確　しました。

> 　　そのアプリを　るのに　っている「　　の　み　わせ」のこと。　　でいう「　っている　　と　　の一　」。　アプリのスタックは `CLAUDE.md` にまとめられています。

	TypeScript	のある安　なJavaScript。画　の　　を　く
	React 19	画　を　　（コンポーネント）で　み　てる
	Material-UI 7	ボタン・　などの　　デザイン
	React Router DOM 7	URLと画　の　応（§10）
	Axios	サーバーとのHTTP　　（§5の `api`）
	Chart.js / FullCalendar	のグラフ　　・カレンダー
	Vitest	フロントの　　テスト
	Django 4.2	サーバー　　の　　み（§7）
	Django REST Framework	API（`APIView`・`Response`。§7）を　る
	Django-Q	バックアップ　の　い　　を　で
	PostgreSQL	データの　　（§8）
	WhiteNoise	CSS・画　の　　（§12）

各　　が「なぜ　ばれたか」を、　　が　　できるよう　　で　価します。

学　しやすさ　　の　　さ（　ったとき　べやすいか）　開　の　さ
のしやすさ　　コミュニティ・　　アプリとの

学	◯	（コンポーネント）の　え　が　感　。ただしstate　の概　は　れが
	◎	界　で　われ、　　も
開	◎	を　み　わせるので、　た画　を　　しやすい（　アプリの　　の一　画　）
	◯	位で　せる。stateの　れを　う　　は　る
	◎	な　　だけ　　画するため画　が　い（§3のSPA）
	◎	界　　。　　も　く　　を　しやすい
	◎	画　・　フォームの　アプリに

学	◯	JavaScriptに「　」を　したもの。　の　だけ覚えることは　える
	◎	みで
開	◯	を　く　　はあるが、エディタの　完が　　で　じて　い
	◎	。　　の安
	—	にJavaScriptに　　されるので　　はJSと
	◎	フロント開　の　　の
	◎	データ　　が　い　アプリで、　による　り違え　　が　く

§5.2で　た `useState<string>("")` の `<string>` のように、「この　は　」と　で　します。すると、うっかり　　を　れようとした　　にエディタが　　でしてくれます。　　ので、　　の　　でも　　が　ります。

学	◯	「　　（　　が　から　う)」で、　まった　き　に　えば　く
	◎	が　く、　　の解　・　が　い
開	◎	画　（§9　）・　・ORM（§8）が　　で、ゼロから　らずに　む
	◎	`views`・`models`・`urls`と　割でファイルが　かれ、どこを　すか　快（　で　感した　り）
	◯	一　な　アプリには　。　な　荷は
	◎	界　に　く　われ安
	◎	データ　・　・　画　が　な　システムに

学	◯	`APIView`・`Serializer`の　が　かれば　できる（§7で　感）
	◎	リクエスト　・JSON　の　化で、　たAPIを　じ　で　ける
	◎	フロント　（§3）でJSON APIを　る　アプリの　核

学	◯	なSQLが　えればよい。Django　（§8のORM）なら　SQLを　く　は　ない
	◎	で、データが壊れにくい。トランザクション（　で　したら巻き　す　み）が　い
	◎	データ・　な　に　い（§7のAdministratorKosuListのような　り　み　）

	◎	オープンソースの　　DB。　　運　に安
	◎	が　　する　アプリのデータ　・　　に　う

> 　　Object-Relational Mapping の　。「データベースの　を、Pythonのコードで　けるようにする　　」。§7.2　の `member.objects.filter(employee_no=...)` のように、SQL（データベース　　）を　かず、Pythonらしい　き　でDBを　　できます。Djangoの　きな　　で、§8のモデルがこのORMの　　です。

アプリのスタックは、　　です。　った　　より、　　を　　した　　な　　といえます。　　にとって　しいのは、　ったときに　　ことです。

　　に、アプリ　　の「　　ファイル」`hozen_another/settings.py`（　202　）の　　を　みます。ここは　　です。　を　違えると　　しなくなることもあるので、　　に扱います。

> 　　アプリの　　フォルダを　め、　　（　）を　外　ファイル（.env）から　み　み、開　モードかどうか、どのホスト　からのアクセスを　すかを　　します。

```
# ① ファイルパスを扱う　具
from pathlib import Path
# ② OS機　(環境変数　)
import os
# ③ .envファイルを　む　具
import environ
# ④ ログを回　(古いものを消す)させる　具
from logging.handlers import RotatingFileHandler



# ⑤ プロジェクトの基準フォルダ(このファイルの2つ上)
BASE_DIR = Path(__file__).resolve().parent.parent

# ⑥ 環境変数　み取り器を準備
env = environ.Env()
# ⑦ .env ファイルを　み　む
env.read_env(os.path.join(BASE_DIR, '.env'))

# ⑧ Djangoの　密　(.envから　む)
SECRET_KEY=env('SECRET_KEY')

# ⑨　 発モードか(.envから真偽値で　む)
DEBUG=env.bool('DEBUG')

# ⑩ 文字コードはUTF-8(日本　が化けない)
DEFAULT_CHARSET = 'utf-8'

# ⑪ アクセスを　可するホスト名ー
ALLOWED_HOSTS = [
    # 本番(Azure)のアドレス
    'hozen-kosu-react-cwaqashkafgbg5e5.japaneast-01.azurewebsites.net',
    #　　発用
    'localhost',
    #　　発用(　分　　のIP)
    '127.0.0.1'
    ]
```

- `BASE_DIR` … プロジェクトの　　となるフォルダ。以　、ファイルの　　を `os.path.join(BASE_DIR, ...)` のように、ここを　　に　み　てます。
- `.env`（ドットエンブ）という　　のが　　なポイントです。

> `.env` は「　　（　・パスワードなど）を　いておく　ファイル」。これを　　、　ファイルに　けて　　します。なぜなら、ソースコードはGit（バージョン　）で　　されることがあり、そこにパスワードを　くと　　するからです。`.env` はGitに　めない（`.gitignore` で　外する）のが　　。`env('SECRET_KEY')` で、その　の　を　み　みます。

- `SECRET_KEY` … Djangoが暗　　（セッションの　など）に　う　　の　。

- `DEBUG` … `True` なら開　モード（　しいエラー画　が　る）、`False` なら　　モード（エラー　　を隠す）。

> ⚠️　　開　モードでは、エラー　に　　（コードやファイルパス）が画　に　　されます。　　でこれが　えると　　の　がかりになり　　です。　の `.env` では　ず `DEBUG=False` にします。

- `ALLOWED_HOSTS` …「このホスト　（　　）宛のアクセスだけ　け　ける」ー　。　　の Azureアドレスと、開　　の `localhost` ・`127.0.0.1` が　　されています。ここに　いホストでアクセスすると　　されます（なりすまし　　）。

```
# ① 有効にする機　(アプリ)の一
INSTALLED_APPS = [
    #      理画　(§9⑦)
    'django.contrib.admin',
    #        機
    'django.contrib.auth',
    #    モデル　理の土台
    'django.contrib.contenttypes',
    #    セッション機　(§7.2⑨で使用)
    'django.contrib.sessions',
    #    メッセージ
    'django.contrib.messages',
    #      的ファイル　理
    'django.contrib.staticfiles',
    #    デザイン　品(　理　の一　)
    'bootstrap4',
    #    日付　択UI
    'bootstrap_datepicker_plus',
    #    本アプリ本体(§6〜§8で　んだkosu)!
    'kosu',
    #      同期処理(Django-Q)
    'django_q',
    #    REST Framework(§7のAPIView　)
    'rest_framework',
    #    CORS(別オリジン　信)対応
    'corsheaders',
]
```

- `　　` … 　　されています。これが　いと、`models.py` の　も `views` の　　も　　されません。
- `　　` … §7で　った `APIView`・`Response` を　　します。
- `　　` … 　の §12.3 のCORS　応。フロント（3000　）とバック（8000　）が　　で　　できるようにします。

```
# ① リクエストが　る「　所」の一　(上から　に　　)
MIDDLEWARE = [
    #　　CORS処理(最初に　くのが定石)
    'corsheaders.middleware.CorsMiddleware',
    #　　セキュリティ
    'django.middleware.security.SecurityMiddleware',
    # セッション(§7のログイン状態保持)
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    #　　CSRF(なりすまし投　)対
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    #　　的ファイル　信(WhiteNoise)
    'whitenoise.middleware.WhiteNoiseMiddleware',
    #　作:現在のリクエスト保持
    'kosu.middleware.clear_session_middleware.CurrentRequestMiddleware',
]

# ② すべてのオリジンからの　信を　可
CORS_ORIGIN_ALLOW_ALL = True
# ③ クッキー(セッション)付き　信を　可
CORS_ALLOW_CREDENTIALS = True
# ④ 明　的に　可するオリジン
CORS_ALLOW_ORIGINS = [
    #　　React　発サーバー
    'http://localhost:3000',
    #　　Django
    'http://localhost:8000',
    'http://127.0.0.1:8000',
]
```

- `MIDDLEWARE`（ミドルウェア）… リクエストがビューに　くまでに　る「　　」の　。

> 　　リクエストとレスポンスの　に　まって、
> 　（セキュリティチェック・セッション　　など）を　う　　み。　　の「 安　　の
> 」のイメージ。リクエストは　から　に各　　を　ってビューへ　き、レスポンスは
> で　ります。

- 　　`CORS`（コルス）の　　。

> Cross-Origin Resource Sharing。ブラウザには「 オリジン（ ドメイン・ ポート）への を、 にさせない」安 があります。 アプリはフロント（3000 ）とバック（8000 ）が なので、このままだと がブロックされます。それを「この み わせは 可」と に解 するのが CORS です。`CORS_ALLOW_CREDENTIALS = True` は「セッションのクッキーも一 にってよい」という、ログイン を つために の です。

```
# ① REST Frameworkの 定
REST_FRAMEWORK = {
    # ページ分割方式
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    #  1ページ20件(一 画 の標準)
    'PAGE_SIZE': 20,
    'DEFAULT_PARSER_CLASSES': (
        #  受信データはJSONとして
        'rest_framework.parsers.JSONParser',
    ),
    'DEFAULT_RENDERER_CLASSES': (
        #  事はJSONで す
        'rest_framework.renderers.JSONRenderer',
    ),
}

# ② セッションをデータベースに保存
SESSION_ENGINE = 'django.contrib.sessions.backends.db'
# ③ セッションの有効期 ( )= 10年
SESSION_COOKIE_AGE = 315360000
# ④ クッキーの 信 囲(なりすまし対 )
SESSION_COOKIE_SAMESITE = 'Lax'
# ⑤ HTTPSのみ 信するか( 発はFalse)
SESSION_COOKIE_SECURE = False
# ⑥ 最上位URL は hozen_another/urls.py(§9)
ROOT_URLCONF = 'hozen_another.urls'
```

- … 一 を20 ずつに けて （ページネーション）。§7.4の `AdministratorKosuList` などで われていた `CustomPagination` の です。`JSONParser`・`JSONRenderer` により「 も もJSON」と まり、§3のJSONやり りが します。

- … セッション （§7.2 で した `login_No` ） を する 。
- … 10 。 「ログインしっぱなし」にする で す（ で何 もログインし す を くため)。
- … 。 すべてのリクエストは、まずここから り けが まります。

```
STATICFILES_DIRS = [
    # ① Reactのビルド成果物(JS/CSS)の場所
    os.path.join(BASE_DIR, 'frontend/build/static'),
]
# Reactのbuild内のindex.htmlをテンプレートとして指定
# ② index.htmlの場所(§9⑩が すもの)
TEMPLATES[0]['DIRS'] = [os.path.join(BASE_DIR, 'frontend/build')]
```

```
# ③ 的ファイルのURL接
STATIC_URL = '/static/'
# ④ 先フォルダ(本番 信用)
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
# ⑤ WhiteNoiseで圧 信
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
# ⑥ アップロードファイルのURL(§6.4で使用)
MEDIA_URL = '/media/'
# ⑦ アップロードファイルの保存先
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

- ここが です。Reactを `npm run build` （`01` で扱う）すると `frontend/build/` に 果 ができ、Djangoはそこの `index.html` （§9 が すもの）と `static/` （JS・CSS）を します。 されています。
- … §11.1の「 」。 でCSS・JS・画 を よく（圧 して） る みです。

```
# ① データベース接　　定
DATABASES = {
    'default': {
        #　　PostgreSQLを使う
        'ENGINE': 'django.db.backends.postgresql',
        #　　DB名(.envから)
        'NAME': os.getenv('DB_NAME', ''),
        #　　ユーザー名(.envから)
        'USER': os.getenv('DB_USER', ''),
        #　　パスワード(.envから)
        'PASSWORD': os.getenv('DB_PASSWORD', ''),
        #　　DBの住所(.envから)
        'HOST': os.getenv('DB_HOST', ''),
        #　　PostgreSQLの標準ポート
        'PORT': '5432',
    }
}

# Django Q の　 定
# ② Django-Q(　同期処理)の　 定
Q_CLUSTER = {
    #　　タスク　 理にDBを使う
    'orm': 'default',
    #　　同時に4つの作業　 で処理
    'workers': 4,
    'recycle': 2000,
    'retry': 2000,
    #　　1タスク最大1800　(30分)
    'timeout': 1800,
}
```

- 　データベース　　。　　（§12.1で　 した　　の　 ）。`postgresql` を　い、　 ポート `5432` で　 します。§8で　 したモデルは、ここで　 したPostgreSQLに　 されます。
- `Q_CLUSTER` … バックアップなど　 のかかる　 を　　する Django-Qの　 。`workers: 4` で4つ　　、1タスク　 30　 。

```
# ③      は日本
LANGUAGE_CODE = 'ja'
# ④ タイムゾーンは東京
TIME_ZONE = 'Asia/Tokyo'
# ⑤ タイムゾーンを意  して時刻を扱う
USE_TZ = True
# ⑥   動IDの型(§8.2の  pk  の型)
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

- ・ の 。エラーメッセージや が けになります。
- `DEFAULT_AUTO_FIELD` … §8.2で れた「各モデルに で く `id`（pk)」の を、 きな（`BigAutoField`）に 。データが に えてもID が 渇しません。

 の `LOGGING`（175〜202 ）は、サーバーの を `web_console.log` というファイルに す です。`RotatingFileHandler` で「ログが1MBを えたら いものを てる」回（`maxBytes: 1024 * 1024 * 1`）にし、ログが に らむのを いでいます。

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    # ログの書式
    'formatters': { 'simple': { 'format': '{message}', 'style': '{', }, },
    'handlers': {
        # web_console.log への出力  定
        'web_console': {
            'level': 'INFO',
            'class': 'logging.handlers.RotatingFileHandler',
            'formatter': 'simple',
            'filename': 'web_console.log',
            # 最大ファイルサイズ (1MB)
            'maxBytes': 1024 * 1024 * 1,
            'backupCount': 1,
            'encoding': 'utf-8',
        },
    },
    'loggers': {
        'views_logger': { 'handlers': ['web_console'], 'level': 'INFO', 'propagate':
False, },
    },
}
```

> これは です。 は `.env` から み（ 洩 ）、`INSTALLED_APPS` で `kosu` を 化し、`CORS` でフロント・バックの ポート を し、`DATABASES` でPostgreSQLにつなぎ、`STATICFILES_DIRS` / `TEMPLATES` でReactのビルド 果 を します。
> のです。

に、プロジェクト の「どこに何があるか」の を します。 ったらここに ってください。

```
hozen-kosu-React/                    ← プロジェクト全体のフォルダ
│
├── manage.py                        ← Djangoの操作コマンド入口(runserver　。01　で使う)
├── .env                             ←　密情報(　・DBパスワード)。Gitに含めない(§12.1)
├── web_console.log                  ← サーバーの動作ログ(§7.1, §12.6)
├── CLAUDE.md                        ← プロジェクト　明書
│
├── hozen_another/                   ← 【プロジェクト　定】Django全体の　定(§9, §12)
│   ├── settings.py                  ←　　定ダイヤル盤(§12)
│   ├── urls.py                      ←　最上位URL　=玄　の交　整理(§9)
│   └── wsgi.py / asgi.py            ←　本番　動の入口
│
├── kosu/                            ← 【アプリ本体】工数　理の実体(§6〜§8)
│   ├── models.py                    ←　データの　　図(§8)
│   ├── urls.py                      ←　アプリ内URL　(§6)
│   ├── views/                       ←　処理(リクエスト→レスポンス)
│   │   ├── main_views.py            ←　　ログイン・各メニュー(§7)
│   │   ├── kosu_views.py            ←　　工数入力の処理
│   │   ├── member_views.py          ←　　人員の処理
│   │   ├── team_views.py            ←　　班員の処理
│   │   ├── def_views.py             ←　　工数区分定　の処理
│   │   ├── inquiry_views.py         ←　　問い合わせの処理
│   │   ├── asynchronous_views.py    ←　　バックアップ　の　同期処理(§6.3)
│   │   └── serializers.py           ←　　データ⇄JSON変換器(§7.1)
│   ├── tasks.py                     ←　Django-Qの　同期タスク本体
│   ├── signals.py                   ←　モデル変更の　動　　(Historyへの　　)
│   ├── middleware/                  ←　　所処理(§12.3)
│   └── utils/                       ←　　助　具(ページ分割　。§7.1)
│
├── staticfiles/                     ← 本番　信用に　　された　的ファイル(§12.5)
├── media/                           ← アップロードファイル(バックアップ　。§12.5)
│
└── frontend/                        ← 【フロントエンド】React側(§5, §10)
    ├── package.json                 ←　フロントの　定・依存ライブラリー　(01　)
    ├── build/                       ←　ビルド成果物(Djangoが　信。§12.5)
    │   ├── index.html               ←　　Reactの土台(§9⑩が　す)
    │   └── static/                  ←　　ビルド済みJS/CSS
    └── src/                         ←　フロントのソースコード(ここを　　する)
        ├── index.tsx                ←　　司令塔=全ルート定　(§10)
        ├── api/axios                ←　　　信　定(§5.1③の　api)
        ├── styles/                  ←　　CSS(　た目。03　)
```

```
│   ├── global.css              ←      全画   共
│   └── 各ページ別/.module.css   ←      画   専用
├── Components/                 ←    共     品(Pagination, Loading   )
├── MainPage/                   ←    ログイン・メニュー(Login.tsx   。§5)
├── KosuPage/                   ←    工数入力の各画
├── MemberPage/                 ←    人員の各画
├── TeamPage/                   ←    班員の各画
├── DefinitionPage/             ←    工数区分定   の各画
├── InquirPage/                 ←    問い合わせの各画
├── AdministratorPage/          ←      理   の各画
└── tests/                      ←    フロントの   動テスト(Vitest)
```

- → `frontend/src/` の該   ページフォルダの `.tsx`、   た   なら `styles/` の `.css`
- →   じく該   ページの `.tsx`
- → `kosu/views/` の該   `_views.py`
- → `kosu/models.py`（   はマイグレーションが   。`01`   ）
- → `kosu/urls.py`（§6）と `hozen_another/urls.py`（§9）
- → `hozen_another/settings.py`（§12）

この   の   に   して、引き   ぎ   に   会いやすいトラブルと   をまとめます。

ログインしても「 員 は がありません」	その が `member` に い（ERROR048）	Django 画 （`/admin/`）で 員を確 ・ 。 化 なら `12345`（§7.3）
ログイン すぐログイン画 に される	セッションが されていない（§7.4で401）	`settings.py` の `CORS_ALLOW_CREDENTIALS=True`（§12.3）と、フロントの がクッキー する か確

フロントから ぶと　エラ （CORS）	さ	
「　可　な がありません_ （ERROR052）	が	
画　は　るが ない	ngo）	ごDjangoを D
画　そのもの	が ド	なら `npm run` 　）
に SECRET_KEY ー	が	イルを　意し `SECRET_KEY`　を _an
データベース ー	が　り 2.6）	L　`DB_USER`・

4. ：`hozen_another/settings.py`（§12）で、(a) 1ページに何　　するか、(b) どのデータベースを　うか、(c)　　は何か、を　して　える（§12.4・§12.6）。

5. ：`Login.tsx`（§5.3　）が　るデータのキー　と、`main_views.py` の `Login`（§7.2　）が　り　すキー　が一　していることを、　　のコードで確　する。

6. ：`index.tsx`（§10.2）で、URL `/member-list` のとき　　される画　の　　を　え、その　　がどのフォルダから `import` されているかを確　する。

7. ：`main_views.py` の `Help` クラス（§7.3）を　み、「　データを　　化したら、どの　　員　　でログインできるか」を　える（ヒント：§7.3　）。これは引き　ぎで　っておくべき　　です。

- 　　は　（　にかけた　の　）の　・　。ログイン／　／　員／　員／　／　い　わせ／　　の各　があり、フォルダがそれに　応する（§1・§13）。
- 　　がいる ——　　（画　）・　　（　）・　　（　）。リクエストとレスポンスでやり　りし、HTTPメソッド（GET/POST/PUT/DELETE）で　を　する（§2）。
- アプリは　　し、　　でデータをやり　りする。　は　　で　まるURL（§3・§9）。
- 　　：`Login.tsx`（　・　）→ `hozen_another/urls.py`（`/api/` →kosu）→ `kosu/urls.py`（`login/` →Login）→ `main_views.py`（`member`　に　い　わせ・　/　　・セッション　）→ JSON　→ `Login.tsx`（`navigate("/")`）→ `index.tsx`（`/` → `MainMenu`　）。（§4〜§10）。
- 　　はデータの　　。1モデル＝1テーブル、1フィールド＝1カラム。`member`（　員。`authority` / `administrator` で3　階　）・`Business_Time_graph`（　）・`kosu_division`（　。1レコードに　50　）が　核。`id`（pk）は　　（§8）。
- 　　は React / TypeScript / Django / DRF / PostgreSQL を　　とした　　。7　で　ると「　が　・　しやすい・　　が　い」のが　み（§11）。
- 　　はアプリの　ダイヤル　。　　は　　から　み（　洩　）、`INSTALLED_APPS` で `kosu` を　　化、`CORS` で　ポート　　を　可、`DATABASES` でPostgreSQL

、`STATICFILES` / `TEMPLATES` でReactのビルド　果　を　　（§12）。

- 　　を　っておけば、「どこを　せばよいか」で　わない。　た　・　　は `frontend/src/`、データ・　　・　　は `kosu/`、　　は `hozen_another/`（§13）。

これで、このアプリの　　が　に　りました。　は、　　に　　のPCでこのアプリを　かせるようにする　　です。

は、「01_　　.md」へ　みます。